

ConciliaEx

Plataforma para Familias con Personas Dependientes

MANUAL DE DESPLIEGUE

Versión 1.0 · Vercel + Supabase

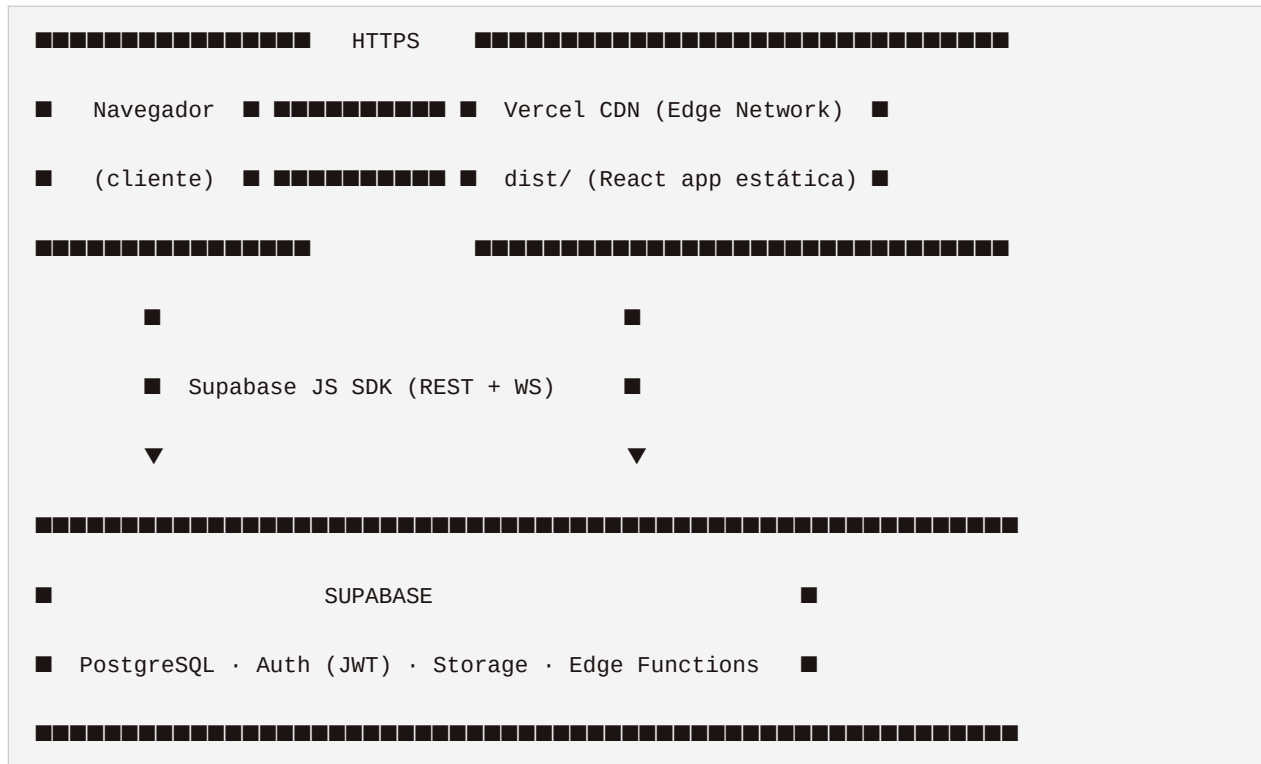
Proyecto Intermodular DAW

IES Albarregas · Mérida, Extremadura

José María Gutiérrez Barrena · 2DAW-B · 2025-2026

1. Visión General de la Arquitectura

ConciliaEx separa completamente el frontend del backend. El frontend es una aplicación web estática desplegada en **Vercel** y el backend es gestionado íntegramente por **Supabase** (PostgreSQL + Auth + Storage).



Servicio	Proveedor	Coste
Hosting frontend	Vercel (Hobby)	Gratuito
Base de datos	Supabase (Free)	Gratuito hasta 500 MB
Auth	Supabase Auth	Gratuito hasta 50 000 MAU
Storage (imágenes)	Supabase Storage	Gratuito hasta 1 GB
Dominio personalizado	Vercel / externo	Opcional

2. Configurar el Proyecto en Supabase

2.1 Crear el proyecto

1. Accede a supabase.com e inicia sesión.
2. Pulsa "New project", elige un nombre (ej: conciliaex), selecciona región Europe West y establece una contraseña segura para la base de datos.
3. Espera ~2 minutos a que el proyecto esté listo.
4. En Project Settings → API anota la URL del proyecto y la anon public key.

2.2 Crear las tablas (SQL Editor)

Abre SQL Editor en el panel de Supabase y ejecuta el siguiente script:

```
-- Tabla perfiles (extiende auth.users)

CREATE TABLE perfiles (

  id          UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,

  email       TEXT NOT NULL,

  nombre      TEXT,

  nombre_entidad TEXT,

  cif         TEXT,

  telefono    TEXT,

  rol         TEXT NOT NULL DEFAULT 'familia',

  activo      BOOLEAN NOT NULL DEFAULT TRUE,

  avatar_url  TEXT,

  created_at  TIMESTAMPTZ DEFAULT NOW()

);

-- Tabla servicios

CREATE TABLE servicios (

  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  entidad_id  UUID REFERENCES perfiles(id) ON DELETE CASCADE,
```

```
nombre      TEXT NOT NULL,

tipo        TEXT NOT NULL,

ubicacion   TEXT,

telefono    TEXT,

descripcion TEXT,

plazas      INT,

activo      BOOLEAN DEFAULT TRUE,

imagen_url  TEXT,

visitas     INT DEFAULT 0,

created_at  TIMESTAMPTZ DEFAULT NOW()

);

-- Tabla solicitudes

CREATE TABLE solicitudes (

    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    servicio_id UUID REFERENCES servicios(id) ON DELETE CASCADE,

    familia_id  UUID REFERENCES perfiles(id) ON DELETE CASCADE,

    entidad_id  UUID REFERENCES perfiles(id),

    nombre_familiar TEXT,

    tipo_necesidad TEXT,

    urgencia    TEXT DEFAULT 'media',

    mensaje     TEXT,

    estado      TEXT DEFAULT 'pendiente',

    mensaje_respuesta TEXT,

    created_at  TIMESTAMPTZ DEFAULT NOW()

);
```

3. Funciones RPC y Trigger de Supabase

3.1 Función admin_delete_user

Permite al administrador eliminar usuarios a través de la service_role. Ejecútala en SQL Editor:

```
CREATE OR REPLACE FUNCTION admin_delete_user(user_id UUID)

RETURNS void LANGUAGE plpgsql SECURITY DEFINER AS $$

BEGIN

    DELETE FROM auth.users WHERE id = user_id;

END;

$$;
```

3.2 Función incrementar_visitas

```
CREATE OR REPLACE FUNCTION incrementar_visitas(servicio_id UUID)

RETURNS void LANGUAGE plpgsql SECURITY DEFINER AS $$

BEGIN

    UPDATE servicios SET visitas = visitas + 1

    WHERE id = servicio_id;

END;

$$;
```

3.3 Trigger handle_new_user

Crea automáticamente un registro en perfiles cada vez que se registra un usuario en Supabase Auth:

```
CREATE OR REPLACE FUNCTION handle_new_user()

RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$

BEGIN

    INSERT INTO perfiles (id, email, rol, nombre, nombre_entidad,

                        cif, telefono)
```

```
VALUES (  
  
    NEW.id,  
  
    NEW.email,  
  
    COALESCE(NEW.raw_user_meta_data->>'rol', 'familia'),  
  
    NEW.raw_user_meta_data->>'nombre',  
  
    NEW.raw_user_meta_data->>'nombre_entidad',  
  
    NEW.raw_user_meta_data->>'cif',  
  
    NEW.raw_user_meta_data->>'telefono'  
  
);  
  
RETURN NEW;  
  
END;  
  
$$;  
  
CREATE OR REPLACE TRIGGER on_auth_user_created  
  
    AFTER INSERT ON auth.users  
  
    FOR EACH ROW EXECUTE FUNCTION handle_new_user();
```

4. Row Level Security (RLS)

Activa RLS en las tres tablas y aplica las siguientes políticas:

```
ALTER TABLE perfiles    ENABLE ROW LEVEL SECURITY;

ALTER TABLE servicios   ENABLE ROW LEVEL SECURITY;

ALTER TABLE solicitudes ENABLE ROW LEVEL SECURITY;


-- PERFILES

CREATE POLICY "Perfil propio" ON perfiles FOR ALL

    USING (auth.uid() = id);


-- SERVICIOS: lectura pública de activos

CREATE POLICY "Ver servicios activos" ON servicios FOR SELECT

    USING (activo = TRUE);

CREATE POLICY "Entidad gestiona sus servicios" ON servicios

    FOR ALL USING (auth.uid() = entidad_id);


-- SOLICITUDES

CREATE POLICY "Familia ve sus solicitudes" ON solicitudes FOR SELECT

    USING (auth.uid() = familia_id);

CREATE POLICY "Familia crea solicitudes" ON solicitudes FOR INSERT

    WITH CHECK (auth.uid() = familia_id);

CREATE POLICY "Entidad ve solicitudes de sus servicios"

    ON solicitudes FOR SELECT

    USING (auth.uid() = entidad_id);

CREATE POLICY "Entidad responde solicitudes" ON solicitudes FOR UPDATE

    USING (auth.uid() = entidad_id);
```

■■ **Atención**

Sin estas políticas el frontend recibirá errores 403. Asegúrate de ejecutar el script completo antes de probar la app.

5. Storage — Buckets e Imágenes

5.1 Crear los buckets

En el panel de Supabase → Storage → New bucket, crea los dos buckets siguientes:

Bucket	Acceso	Uso
avatars	Public	Fotos de perfil de usuarios
servicios	Public	Imágenes de los servicios publicados

5.2 Políticas de Storage

```
-- avatars: el usuario solo sube/actualiza su propio avatar

CREATE POLICY "Avatar propio" ON storage.objects FOR INSERT

    WITH CHECK (bucket_id='avatars' AND auth.uid()::text = (storage.foldername(name))[1]);

CREATE POLICY "Ver avatares" ON storage.objects FOR SELECT

    USING (bucket_id='avatars');

-- servicios: la entidad propietaria gestiona sus imágenes

CREATE POLICY "Subir imagen servicio" ON storage.objects FOR INSERT

    WITH CHECK (bucket_id='servicios' AND auth.role()='authenticated');

CREATE POLICY "Ver imágenes servicios" ON storage.objects FOR SELECT

    USING (bucket_id='servicios');
```

6. Entorno Local (Desarrollo)

1. **Clonar el repositorio:** `git clone https://github.com/GutiBarr/DAW_Jose_Maria_Gutierrez.git && cd DAW_Jose_Maria_Gutierrez`
2. **Instalar dependencias:** `npm install`
3. **Crear el fichero de entorno:** `cp .env.example .env` (o crea `.env` manualmente)
4. **Rellenar las variables:** añade `VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY` con los valores de tu proyecto Supabase.
5. **Arrancar el servidor de desarrollo:** `npm run dev` → abre `http://localhost:5173`

6.1 Fichero .env

```
VITE_SUPABASE_URL=https://xxxxxxxxxxxxxxxxx.supabase.co  
  
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

■ Consejo

El fichero `.env` está en `.gitignore` y nunca se sube al repositorio.
Usa `.env.example` (sin valores reales) como plantilla para el equipo.

6.2 Scripts disponibles

Comando	Descripción
<code>npm run dev</code>	Servidor de desarrollo con HMR en <code>localhost:5173</code>
<code>npm run build</code>	Build de producción en la carpeta <code>dist/</code>
<code>npm run preview</code>	Previsualización local del build de producción

7. Despliegue en Vercel

7.1 Preparar el fichero vercel.json

Crea vercel.json en la raíz del repositorio para que el enrutamiento de React Router funcione correctamente en producción:

```
{  
  
  "rewrites": [  
  
    { "source": "/((?!api/.*)*)", "destination": "/index.html" }  
  
  ]  
  
}
```

7.2 Importar el proyecto en Vercel

1. Accede a vercel.com e inicia sesión con tu cuenta de GitHub.
2. Pulsa "Add New → Project" e importa el repositorio GutiBarr/DAW_Jose_Maria_Gutierrez.
3. Framework Preset: Vite (se detecta automáticamente).
4. Build Command: npm run build (valor por defecto).
5. Output Directory: dist (valor por defecto).
6. Pulsa "Deploy". El primer despliegue tarda ~60 segundos.

7.3 CI/CD automático

Cada push a la rama main de GitHub dispara un nuevo despliegue en Vercel automáticamente. Los pull requests generan preview deployments con URL temporal.

8. Variables de Entorno en Vercel

1. En el dashboard de Vercel, selecciona el proyecto y ve a Settings → Environment Variables.
2. Añade las dos variables con sus valores reales:

Variable	Entornos	Valor
VITE_SUPABASE_URL	Production, Preview, Dev	URL de tu proyecto Supabase
VITE_SUPABASE_ANON_KEY	Production, Preview, Dev	Anon public key de Supabase

■■ Atención

Tras añadir las variables pulsa "Redeploy" para que el build las recoja.

Sin estas variables la app se desplegará pero fallará al conectar con Supabase.

9. Crear el Primer Usuario Admin

1. Registra un nuevo usuario desde la app con cualquier correo y contraseña.
2. Accede al panel de Supabase → Table Editor → perfiles.
3. Localiza el registro del usuario recién creado y edita el campo rol cambiándolo a admin.
4. Guarda el cambio. El usuario ahora tiene acceso al panel /admin.

■ Consejo

Alternativamente puedes ejecutar en SQL Editor: `UPDATE perfiles SET rol = 'admin' WHERE email = 'tu@email.com';`

10. Dominio Personalizado (Opcional)

1. En Vercel → Settings → Domains, pulsa "Add" e introduce tu dominio (ej: conciliaex.es).
2. Vercel te mostrará los registros DNS a añadir en tu proveedor de dominio.
3. Añade el registro CNAME (o A) en el panel DNS del registrador.
4. Vercel provisionará automáticamente el certificado TLS (Let's Encrypt) en ~5 minutos.

10.1 Actualizar las URLs de Supabase

Si usas dominio personalizado, actualiza en Supabase → Authentication → URL Configuration:

- **Site URL:** `https://conciliaex.es`
- **Redirect URLs:** `https://conciliaex.es/**`, `https://conciliaex.es/nueva-password`

11. Verificación Post-Despliegue

Realiza las siguientes comprobaciones tras cada despliegue a producción:

#	Verificación	Resultado esperado
1	Abrir la URL de producción	Carga la Landing correctamente
2	Navegar a /catalogo	Se muestran los servicios activos
3	Navegar a /login directamente	Aparece la pantalla de login (no 404)
4	Navegar a /familia/dashboard sin sesión	Redirige a /login
5	Registrar cuenta de familia	Correo de confirmación recibido
6	Iniciar sesión con familia	Redirige al dashboard de familia
7	Solicitar plaza en un servicio	Solicitud aparece en el dashboard
8	Iniciar sesión con entidad	Redirige al dashboard de entidad
9	Publicar un servicio	Aparece en el catálogo
10	Responder una solicitud	Estado cambia en el panel de familia
11	Iniciar sesión con admin	Acceso a /admin correcto
12	Cambiar rol de usuario desde admin	Rol actualizado en tabla perfiles
13	Subir foto de perfil	Imagen visible en Storage > avatars
14	Cambiar idioma ES/EN	Toda la UI cambia de idioma
15	Activar modo oscuro	Tema oscuro aplicado correctamente

12. Mantenimiento y Operaciones

12.1 Rollback en Vercel

Si un despliegue presenta problemas, ve a Vercel → Deployments, selecciona el despliegue anterior estable y pulsa "Promote to Production".

12.2 Logs en tiempo real

- **Vercel:** Dashboard → proyecto → Functions / Logs para errores de build y runtime.
- **Supabase:** Logs → API para ver todas las peticiones REST y errores de RLS.

12.3 Backups de la base de datos

El plan Free de Supabase realiza backups diarios automáticos con retención de 7 días. Para restaurar, ve a Supabase → Database → Backups.

12.4 Actualizar dependencias

```
npm outdated           # ver versiones desactualizadas

npm update              # actualizar según rangos de package.json

npm-check-updates -u    # actualizar a últimas versiones (con cuidado)

npm install             # instalar versiones actualizadas

npm run build           # verificar que el build sigue funcionando
```


13. Solución de Problemas Comunes

Síntoma	Pantalla en blanco al cargar
Causa probable	Variables de entorno no configuradas en Vercel
Solución	Añadir VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY en Settings → Env Vars y redespargar.

Síntoma	Error 404 al recargar una ruta
Causa probable	vercel.json ausente o mal configurado
Solución	Verificar que vercel.json con el rewrite existe en la raíz del repo.

Síntoma	Error 403 al leer/escribir datos
Causa probable	Políticas RLS no creadas o incorrectas
Solución	Revisar y ejecutar el script de RLS en Supabase SQL Editor.

Síntoma	El trigger no crea el perfil
Causa probable	Trigger handle_new_user no instalado
Solución	Ejecutar el script del trigger en SQL Editor.

Síntoma	Imágenes no se muestran
Causa probable	Buckets privados o políticas de Storage incorrectas
Solución	Confirmar que los buckets son públicos y las políticas SELECT están creadas.

Síntoma	Correo de confirmación no llega
Causa probable	Email de Supabase en spam o SMTP no configurado
Solución	Revisar carpeta spam; configurar SMTP personalizado en Auth → SMTP Settings.

Síntoma	Sesión no persiste al recargar
Causa probable	localStorage bloqueado por navegador
Solución	Desactivar modo privado o extensiones que bloqueen localStorage.

14. Referencia Rápida

14.1 Comandos frecuentes

```
# Desarrollo local

npm install && npm run dev

# Build de producción

npm run build

# Promover despliegue anterior en Vercel (desde CLI)

vercel promote <deployment-url>

# Consultar logs en Vercel CLI

vercel logs <deployment-url>
```

14.2 URLs clave del proyecto

Recurso	URL
Repositorio GitHub	https://github.com/GutiBarr/DAW_Jose_Maria_Gutierrez
Dashboard Vercel	https://vercel.com/dashboard
Dashboard Supabase	https://supabase.com/dashboard
Documentación Supabase	https://supabase.com/docs
Documentación Vercel	https://vercel.com/docs

— Fin del Manual de Despliegue —